

THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING AND INFORMATICS

NAME:	PHIL SAMAKAYI (2021439488)
	BLESSING YABE (2021417034)
COURSE CODE:	CSC 4004
COURSE:	PROJECTS
PROJECT TITLE:	STUDYPLANNERAPP
SUPERVISOR:	MR. MARTIN PHIRI

Inception — Vision and Scope

1. Product vision

StudyPlannerApp is a web-based study management application intended to help university students organize individual study time and find academically complementary peers for collaborative study.

The implemented core idea is:

A student records subject-specific strengths and weaknesses, provides recurring weekly availability, and requests an AI-assisted peer match for a selected subject.

The matching implementation uses a vector representation of the requesting student's weakness/strength profile and candidate students' complementary profile, computes cosine similarity, applies an availability penalty when weekly slots do not overlap, ranks the candidates, and persists the highest-ranked pairing.

2. Primary stakeholders

3. Current actors

Student

The primary actor. A student can authenticate, manage profile information, read subjects, maintain self-assessments and availability, create personal study sessions, request/view matches, schedule and complete group-study sessions, and submit feedback.

Administrator

An administrative user/staff member can manage the subject catalogue. The feedback metrics endpoint is also accessible to administrators/staff.

Matching engine

The matching engine is **not** modelled as a domain actor in the domain model. It is an internal application/service component invoked by the matching controller.

4. Current functional scope

- Account registration and JWT authentication
- Student profile retrieval/update
- Subject catalogue
- Self-assessment CRUD and bulk submission
- Weekly recurring availability
- Personal study sessions
- AI peer matching
- Match history for the authenticated student
- Group-study session scheduling
- Group-study session completion
- Post-session feedback
- Aggregated feedback metrics

5. Planned features not evidenced as implemented

These remain future scope unless the implementation is extended:

- Transcript upload
- Automatic transcript parsing
- Academic profile generation from transcript grades
- Manual confirmation/correction of extracted transcript data
- Explicit opt-in/opt-out workflow for GroupStudy
- Invitation lifecycle with accept/decline
- Real-time chat/video
- LMS integration
- GPA calculation
- Payment functionality

6. Key quality concerns

1. **Security** — authenticated access and ownership restrictions must remain enforced.
2. **Matching quality** — recommendations should be useful and should be evaluated through feedback.
3. **Data integrity** — assessment uniqueness, valid 1–5 ratings, valid time ranges and participant restrictions are important.
4. **Maintainability** — the matching algorithm should remain isolated from controllers so it can evolve independently.
5. **Usability** — the core student workflow should be understandable without developer assistance.

7. Success criteria for the current implementation

A practical acceptance baseline is:

- Student can register and log in.
- Student can maintain profile, subjects, self-assessments and weekly availability.
- Student can create personal study sessions.
- Student can request a subject-specific peer match.
- The system returns a persisted match when a candidate exists.
- Student can schedule and complete a matched group-study session.
- Feedback is only accepted for completed sessions and only from participants.
- Feedback metrics can be calculated from submitted ratings.

Inception — Use Case Model

1. Use-case identification

The following use cases are grounded in the uploaded implementation.

2. Core use-case priorities

Highest priority

- UC-02 Log In

- UC-05 Manage Self-Assessment
- UC-06 Manage Weekly Availability
- UC-08 Request AI Peer Match
- UC-09 View My Matches
- UC-10 Schedule Group Study Session
- UC-12 Submit Post-Session Feedback

These represent the core differentiating workflow.

Supporting priority

- UC-01 Register Account
- UC-03 Manage Profile
- UC-04 View Subject Catalogue
- UC-07 Manage Personal Study Session
- UC-11 Complete Group Study Session
- UC-13 View Feedback Metrics

3. Use-case relationships

Keep UML relationships deliberately limited, consistent with Larman's advice not to replace useful use-case text with excessive `include`/`extend` relationships.

- **Schedule Group Study Session** requires the student to have a match, so viewing/selecting an existing match is a prerequisite.
- **Submit Post-Session Feedback** requires a completed session.
- **Request AI Peer Match** results in a match being available for later viewing.

The detailed behavioural rules belong in the use-case descriptions and operation contracts, not only in the diagram.

4. Fully dressed core use case — Request AI Peer Match

Goal: Find a suitable study peer for a selected subject.

Preconditions

6. Student is authenticated.
7. Student has a self-assessment for the selected subject.
8. The subject exists.

Main success scenario

9. Student selects a subject.
10. Student requests a peer match.
11. System retrieves the student's self-assessment.
12. System finds candidate self-assessments for the same subject.
13. System obtains recurring availability for the requesting student and candidates.
14. Matching engine constructs the comparison vectors.
15. Matching engine calculates cosine similarity.
16. Matching engine applies the availability factor.
17. Candidates are sorted by final relevance score.

18. System selects the highest-ranked candidate.
19. System creates a Match.
20. System creates MatchParticipant records for the requesting student and selected peer.
21. System returns the created Match.

Alternative

- If no self-assessment or candidates exist, the system returns no match rather than creating a Match record.

5. Fully dressed core use case — Submit Post-Session Feedback

Preconditions

22. Student is authenticated.
23. Student participated in the referenced group-study session.
24. The session is completed.
25. Student has not already submitted feedback for that session.

Main success scenario

26. Student selects a completed session.
27. Student submits a rating and optional comment.
28. System validates the session status.
29. System validates participation.
30. System validates that duplicate feedback does not exist.
31. System persists the feedback.
32. System returns confirmation.

Inception — Risks and Initial Iteration Plan

1. Major risks

2. RUP-style iteration strategy

Inception iteration

Establish:

- vision
- actors
- use cases
- major risks
- initial architecture
- success criteria

Elaboration iteration

Prove the architecture through the architecturally significant scenario:

Request AI Peer Match.

Deliver:

- domain model
- SSD
- operation contracts
- design class view
- architecture
- persistence model

Construction iteration 1

Implement and stabilize:

- registration/login
- profile
- subjects
- self-assessment

Construction iteration 2

Implement and stabilize:

- availability
- personal study sessions
- AI matching

Construction iteration 3

Implement and stabilize:

- match persistence/history
- group-study session scheduling/completion
- feedback

Transition

Run integrated acceptance tests, verify security/ownership rules, collect match-quality feedback, fix defects and prepare the final release.

Elaboration — Conceptual Domain Model

1. Purpose

The domain model is a conceptual view of the problem domain, not a software class diagram. It identifies important concepts, attributes and associations without placing controller/service methods into the model.

This follows the distinction emphasized by Craig Larman: a domain model should represent conceptual classes from the domain rather than software artifacts.

2. Conceptual classes

Student
Represents the university student using the application.

StudentProfile
Stores student number, programme and year of study associated one-to-one with a user.

Subject
Represents a course/subject in the catalogue.

SelfAssessment
Represents the student's perceived strength and weakness in one subject. The implementation constrains both scores to 1–5 and permits at most one assessment per student/subject.

ScheduleSlot
Represents a recurring weekly availability block.

StudySession
Represents an individual personal study session linked to a subject.

Match
Represents the persisted result of a matching request, with a relevance score.

MatchParticipant
Represents participation of a Student in a Match. It is a persistence association/junction concept needed to represent the many-to-many relationship.

GroupStudySession
Represents a scheduled collaborative session created from a Match. Its status can be scheduled, completed or cancelled.

Feedback
Represents post-session feedback submitted by a participating student.

3. Important associations

- Student 1 — 1 StudentProfile
- Student 1 — 0..* SelfAssessment
- Subject 1 — 0..* SelfAssessment
- Student 1 — 0..* ScheduleSlot
- Student 1 — 0..* StudySession
- Subject 1 — 0..* StudySession
- Student 0..* — 0..* Match through MatchParticipant
- Match 1 — 0..1 GroupStudySession
- Student 1 — 0..* Feedback
- GroupStudySession 1 — 0..* Feedback

4. Domain-model observations

The current implementation does **not** contain a separate `StudyTimetable` class. A timetable is effectively represented by the collection of `StudySession` objects belonging to a student.

Likewise, the AI matching engine is intentionally excluded from the conceptual domain model. It is a software responsibility rather than a conceptual entity in the student's study-planning domain.

Elaboration — System Sequence Diagrams

The SSDs show system events at the boundary of the system. They are intentionally less detailed than a design sequence diagram.

SSD-01 — Request AI Peer Match

Actor: Student

33. `requestMatch(subjectId)`
34. System authenticates the request.
35. System retrieves the student's assessment for the subject.
36. System retrieves candidate assessments.
37. System retrieves recurring availability.
38. Matching engine scores candidates.
39. System persists the highest-ranked Match and its participants.
40. System returns the Match.

Alternative: no assessment/candidates → return no-match response.

Matching rule currently implemented

Request vector:

`[requesting weakness score, 6 - requesting strength score]`

Candidate vector:

`[candidate strength score, 6 - candidate weakness score]`

Compatibility:

`cosine_similarity(request_vector, candidate_vector)`

Availability:

- overlapping exact recurring `(day, start, end)` slot → factor `1.0`
- no overlap → factor `0.75`
- if either side has no slots → treated as overlapping by the current implementation

Final relevance:

`round(similarity × availability_factor, 2)`

The top candidate is persisted as the Match.

SSD-02 — Update Weekly Availability

41. Student selects weekly slots.
42. UI sends bulk update.
43. System authenticates the student.
44. Existing slots for that student are deleted.
45. Submitted slots are validated.
46. New slots are created.
47. System returns the saved slots.

SSD-03 — Submit Post-Session Feedback

48. Student submits session, rating and comment.
49. System verifies the session is completed.
50. System verifies the student participated.
51. System verifies duplicate feedback does not already exist.
52. System creates Feedback.
53. System returns confirmation.

SSD-04 — Schedule Group Study Session

54. Student selects a Match.
55. Student supplies scheduled time.
56. System verifies the student is a participant in the Match.
57. System creates the GroupStudySession.
58. System returns the scheduled session.

Elaboration — Operation Contracts

Operation contracts describe important system operations in terms of preconditions, postconditions and system state changes.

OC-01 — requestMatch(subjectId)

Cross-reference: UC-08 / SSD-01

Preconditions

- Current user is authenticated.
- `subjectId` identifies an existing Subject.
- Current user has a SelfAssessment for that subject.

Postconditions on success

- A Match is created.
- Match.relevance_score equals the selected candidate's final relevance score.
- Two MatchParticipant associations exist: requesting student and selected candidate.
- Match is retrievable through the requesting student's match list.

Postconditions when no candidate exists

- No Match is created.
- A no-candidate response is returned.

OC-02 — bulkUpdateSlots(slots)

Cross-reference: UC-06 / SSD-02

Preconditions

- Current user is authenticated.
- `slots` is a list.
- Every slot has a valid day/time combination.
- `end_time > start_time`.

Postconditions

- All previous ScheduleSlot rows for the current student are removed.
- One ScheduleSlot is created for each valid submitted slot.
- Returned collection contains the student's new availability.

OC-03 — scheduleGroupStudySession(match, scheduledTime)

Cross-reference: UC-10 / SSD-04

Preconditions

- Current user is authenticated.
- Current user participates in the supplied Match.

Postconditions

- A GroupStudySession is created for the Match.
- Scheduled time is stored.
- Initial status is `scheduled`.

OC-04 — completeGroupStudySession(session)

Cross-reference: UC-11

Preconditions

- Current user can retrieve the session through their participating Match.

Postconditions

- Session.status becomes `completed`.
- Feedback becomes eligible for that session.

OC-05 — submitFeedback(session, rating, comment)

Cross-reference: UC-12 / SSD-03

Preconditions

- Current user is authenticated.
- Session status is `completed`.
- Current user is a participant in the session's Match.

- Current user has not already submitted feedback for that session.
- Rating is 1–5.

Postconditions

- A Feedback record exists linked to the student and session.
- Feedback contains the supplied rating and optional comment.

Elaboration — Design Class View

1. Purpose

Unlike the domain model, this design view represents software responsibilities. It therefore contains controllers/viewsets, the matching service, repositories/data-access responsibilities and domain/data classes.

2. Controller responsibilities

AccountController

- register
- login/token issuance
- refresh token
- retrieve/update authenticated profile

AssessmentController

- list/create own assessments
- retrieve/update/delete own assessment
- bulk submit assessments

SchedulingController

- list/manage own recurring availability
- bulk update availability
- manage own personal study sessions

MatchController

- request a match
- list matches belonging to the authenticated student

GroupStudySessionController

- schedule group session for a participant's match
- list sessions for participant
- mark session completed

FeedbackController

- create/read/update/delete feedback within authorization rules
- calculate aggregate feedback metrics

3. MatchingService

The current `matching.engine.generate_peer_matches()` is the most important application-service responsibility.

It:

59. retrieves the requester's assessment;
60. builds the requester vector;
61. retrieves candidate assessments for the same subject;
62. obtains availability;
63. calculates cosine similarity;
64. applies the availability factor;
65. sorts results;
66. returns the top N candidates.

Keeping this responsibility outside the controller follows the principle of assigning a complex domain/application responsibility to a focused service rather than making the controller perform the algorithm.

4. Design principles visible in the implementation

- **Controller:** Django REST viewsets coordinate requests and delegate work.
- **Information Expert:** models/serializers validate and expose information they own.
- **Protected Variations:** the matching algorithm is isolated in `matching.engine`, making changes to scoring less likely to affect the API controller.
- **Creator:** the matching controller creates the Match and MatchParticipant records as part of the match-request use case.
- **Low Coupling:** matching, scheduling, assessments and accounts are separated into Django apps.

Elaboration — Architecture and Design Rationale

1. Architectural style

The implementation is a web client + REST API architecture:

React/Vite presentation → Django REST application → relational database

The matching engine is an in-process application component inside the Django backend.

2. Presentation tier

React components provide:

- dashboard
- authentication screens
- self-assessment form
- availability grid
- peer-match request
- feedback interaction

The Axios API service attaches JWT access tokens to requests and attempts refresh on HTTP 401.

3. Application tier

Django REST Framework exposes endpoint groups for:

- `/api/accounts/`
- `/api/auth/`
- `/api/subjects/`
- `/api/assessments/`
- `/api/scheduling/`
- `/api/matching/`

4. Matching component

The matching engine is currently **in-process**, not a separate deployed microservice.

This is an important correction to the earlier conceptual documentation: the uploaded code imports the matching engine directly from `matching.engine`.

5. Data tier

The repository contains SQLite for local development. The project's earlier design material described MySQL as a production target, but the uploaded implementation/configuration should be treated as the current authority for what is actually deployed.

6. Security

- JWT authentication
- authenticated-only access for student data
- ownership filtering on assessments, schedules and personal sessions
- participant checks before scheduling group sessions
- participant and completion checks before feedback
- unique constraints for assessment and feedback duplication

7. Architectural risk

The matching engine currently performs database access itself. A future refinement could introduce explicit repository interfaces if the team needs easier unit testing or if the matching algorithm is extracted into a separate service.

Elaboration — Requirements / Design Traceability

Larman/RUP artifact trace

Construction — Iteration Plan

Construction should proceed in short, use-case-driven iterations rather than implementing the entire system in one pass.

Iteration C1 — Accounts and Catalogue

Goal: Establish the authenticated student foundation.

Scope:

- registration
- login/token handling
- profile
- subject catalogue
- authorization

Exit criteria:

- student can register/login;
- authenticated profile can be retrieved;
- subjects can be listed;
- only authorized staff/admin users can modify subjects.

Iteration C2 — Academic Self-Assessment

Scope:

- list/create/update/delete assessments
- bulk submission
- validation
- one assessment per student/subject

Exit criteria:

- invalid scores are rejected;
- duplicate student/subject assessment is prevented;
- student cannot access another student's assessment.

Iteration C3 — Scheduling

Scope:

- weekly recurring availability
- personal study sessions

Exit criteria:

- availability can be replaced through bulk update;
- invalid time ranges are rejected;
- personal sessions are isolated to the authenticated student.

Iteration C4 — AI Matching

Scope:

- subject-specific candidate selection
- cosine similarity

- availability factor
- ranking
- Match persistence
- participant associations

Exit criteria:

- a valid request returns the best available candidate;
- no-candidate path is handled cleanly;
- match is persisted;
- only the student's own matches are returned.

Iteration C5 — Group Study and Feedback

Scope:

- schedule session
- complete session
- feedback
- metrics

Exit criteria:

- only match participants can schedule;
- completed status is required for feedback;
- only participants can submit feedback;
- duplicate feedback is rejected;
- metrics are calculated.

Construction — Implementation Traceability

This artifact connects the design responsibilities to the current project modules.

Important design-to-code discrepancy

Earlier UML material in the project describes an AI service and some planned entities/features that are more elaborate than the uploaded code. The current implementation should be used when defending the project in a presentation.

For example, the current matching request creates the **top match directly**. It does not persist a list of all five ranked candidates as Match objects.

Similarly, the current implementation does not implement transcript parsing or an invitation state machine.

Transition — Acceptance and Validation Plan

1. Purpose

Transition verifies that the integrated StudyPlannerApp satisfies the core use cases and that the architecturally significant matching workflow behaves correctly.

2. Acceptance scenarios

AT-01 Registration/Login

Given a new student

When valid registration data is submitted

Then the account is created and the student can obtain JWT tokens.

AT-02 Self-assessment

Given an authenticated student

When the student submits strength and weakness values from 1 to 5

Then the assessment is stored for that student and subject.

AT-03 Availability

Given an authenticated student

When weekly availability is saved

Then the student's previous availability is replaced by the submitted slots.

AT-04 Match success

Given two students with assessments for the same subject

When one student requests a match

Then a Match is created with two participants and a relevance score.

AT-05 No-match path

Given a student with no assessment or no candidate

When the student requests a match

Then the system returns an informational no-match response and does not create an invalid Match.

AT-06 Group study authorization

Given a Match

When a participant schedules a session

Then the GroupStudySession is created.

When a non-participant attempts the same action

Then validation rejects the operation.

AT-07 Feedback

Given a completed GroupStudySession

When a participating student submits a rating from 1 to 5

Then Feedback is created.

When the session is not completed

Then feedback is rejected.

When the student already submitted feedback

Then the duplicate is rejected.

3. Matching-quality validation

Collect post-session ratings and calculate:

- total reviews
- average rating
- positive reviews (`rating >= 4`)
- perceived relevance rate

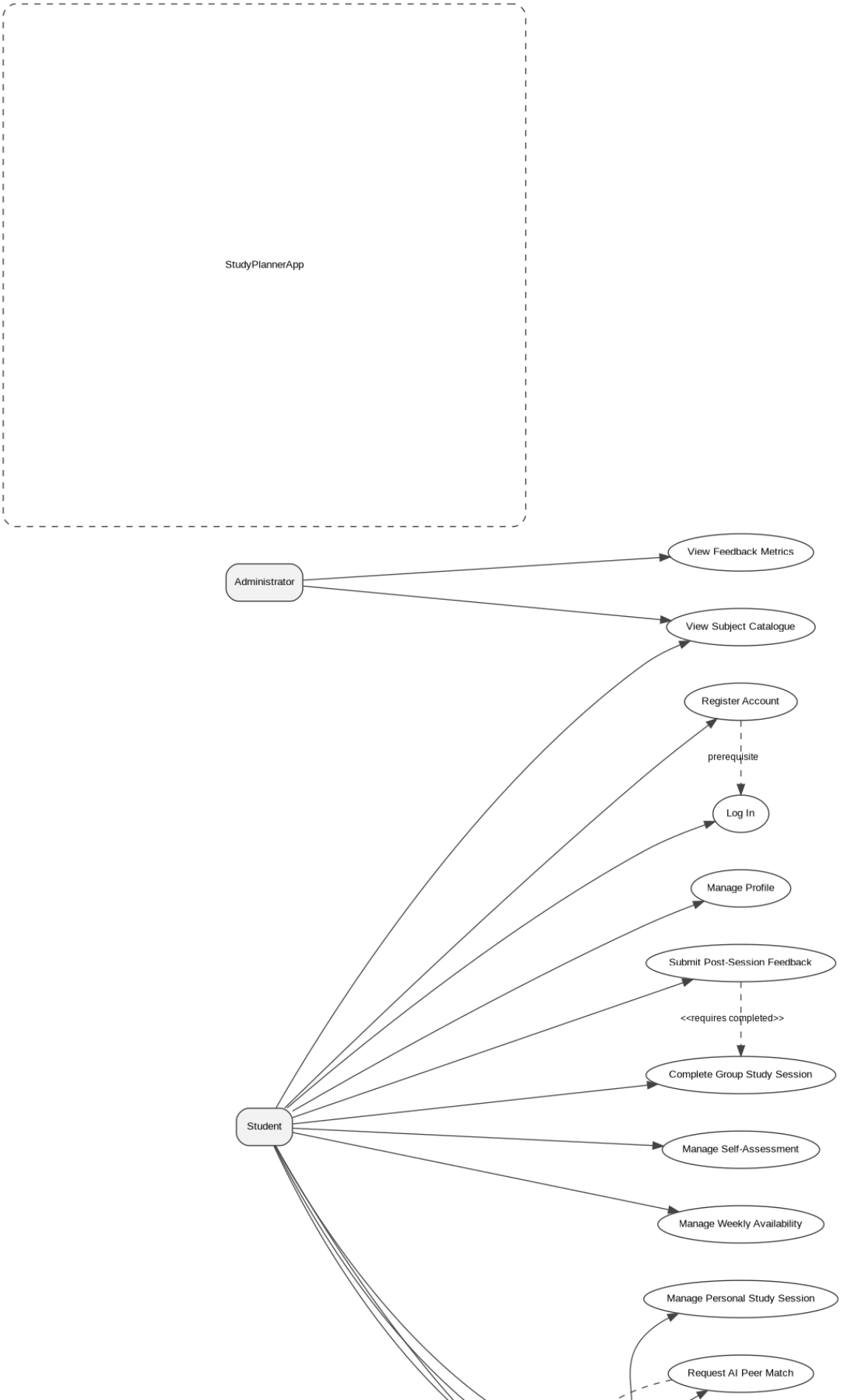
The current metrics endpoint uses a 75% positive-review threshold as its objective indicator.

4. Transition checklist

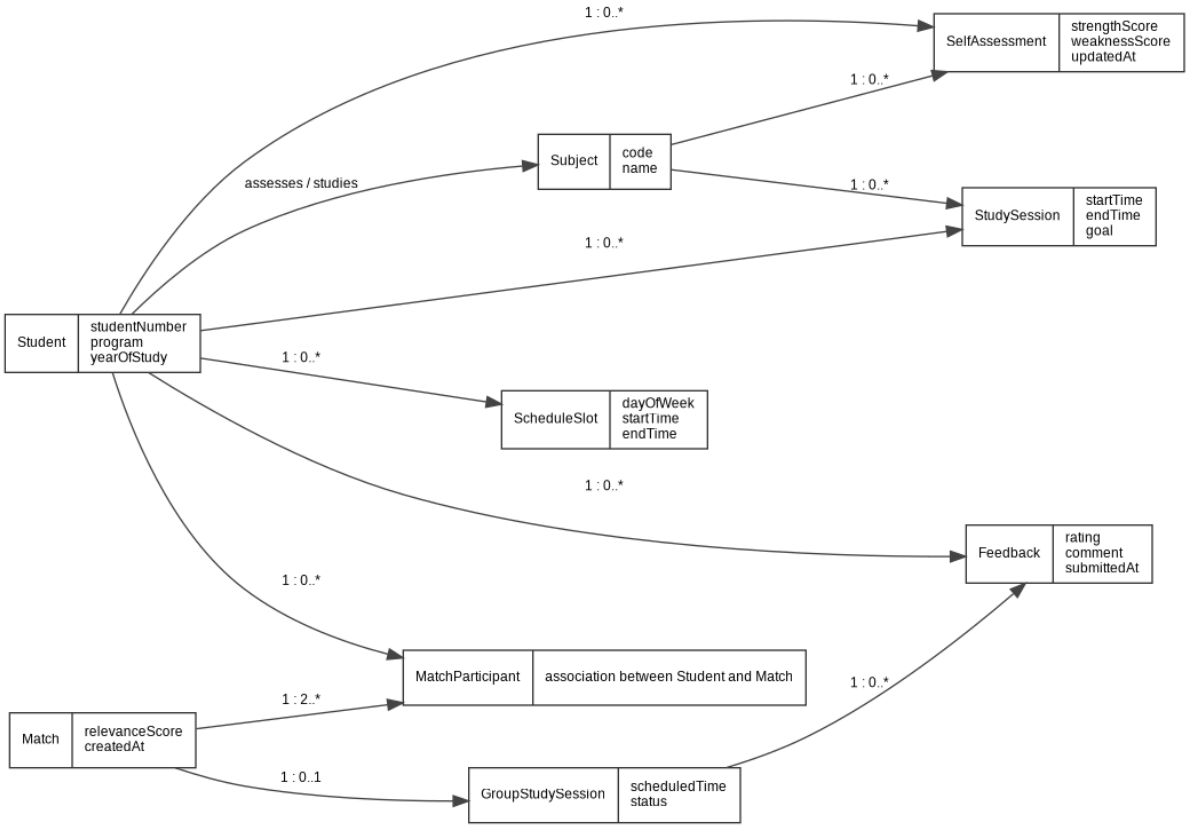
- [ ] Run database migrations on clean environment.
- [ ] Verify registration/login.
- [ ] Verify per-user authorization.
- [ ] Verify subject administration.
- [ ] Verify assessment constraints.
- [ ] Verify availability validation.
- [ ] Verify matching success and no-match paths.
- [ ] Verify group-study participant restrictions.
- [ ] Verify completion and feedback rules.
- [ ] Verify feedback metrics.
- [ ] Verify frontend/API integration.
- [ ] Record known defects and limitations.
- [ ] Confirm future-scope features are not presented as implemented.

UML Diagram Figures

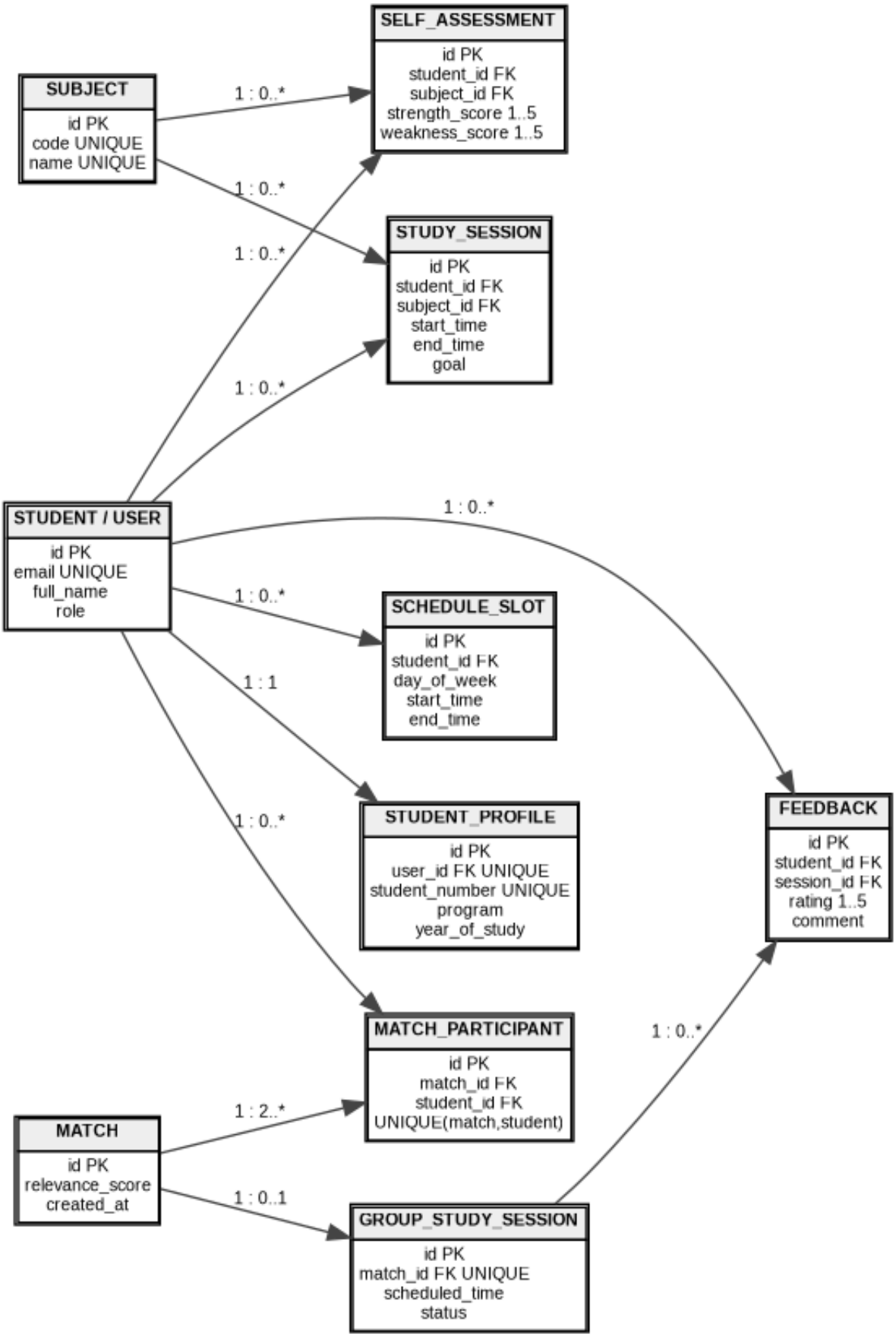
01 Use Case Model


StudyPlannerApp
Administrator
View Feedback Metrics
View Subject Catalogue
Register Account
prerequisite
Log In
Manage Profile
Submit Post-Session Feedback
<<requires completed>>
Complete Group Study Session
Student
Manage Self-Assessment
Manage Weekly Availability
Manage Personal Study Session
Request AI Peer Match

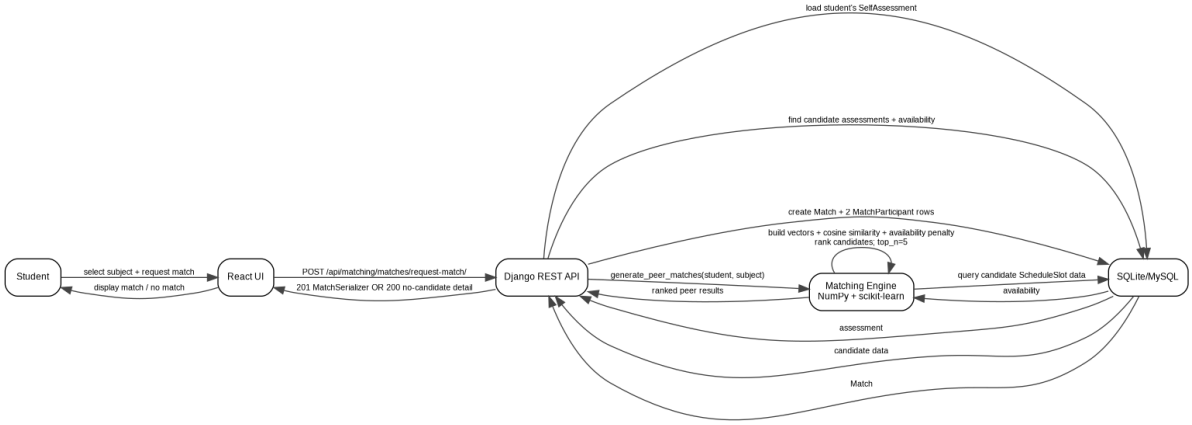
02 Domain Model



03 Entity Relationship Model


SELF_ASSESSMENT
id PK
student_id FK
subject_id FK
strength_score 1..5
weakness_score 1..5
SUBJECT
id PK
code UNIQUE
name UNIQUE
1 : 0..*
1 : 0..*
STUDY_SESSION
id PK
student_id FK
subject_id FK
start_time
end_time
goal
1 : 0..*
1 : 0..*
STUDENT / USER
id PK
email UNIQUE
full_name
role
1 : 0..*
1 : 0..*
SCHEDULE_SLOT
id PK
student_id FK
day_of_week
start_time
end_time
1 : 1
STUDENT_PROFILE
id PK
user_id FK UNIQUE
student_number UNIQUE
program
year_of_study
1 : 0..*
FEEDBACK
id PK
student_id FK
session_id FK
rating 1..5
comment
MATCH_PARTICIPANT
id PK
match_id FK
student_id FK
UNIQUE(match,student)
1 : 0..*
1 : 2..*
MATCH
id PK
relevance_score
created_at
1 : 0..1
GROUP_STUDY_SESSION
id PK
match_id FK UNIQUE
scheduled_time
status

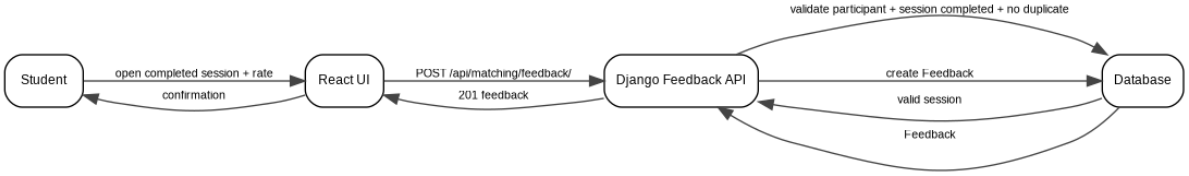
04 SSD AI Match



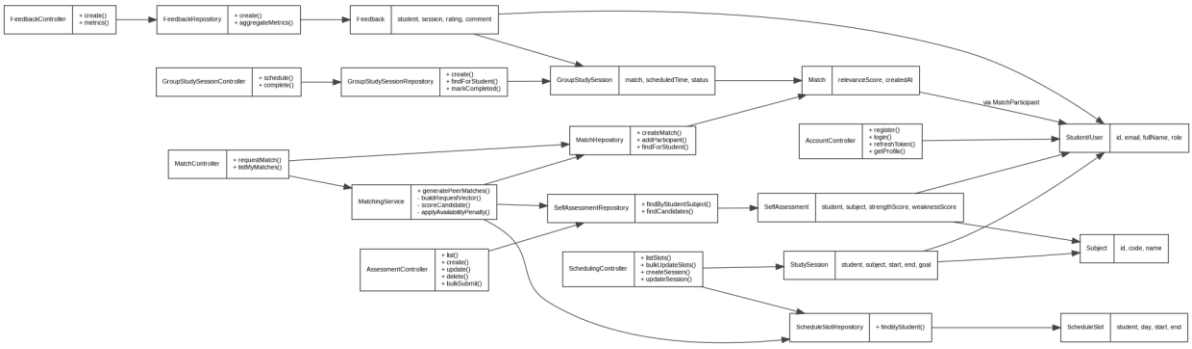
05 SSD Availability Schedule



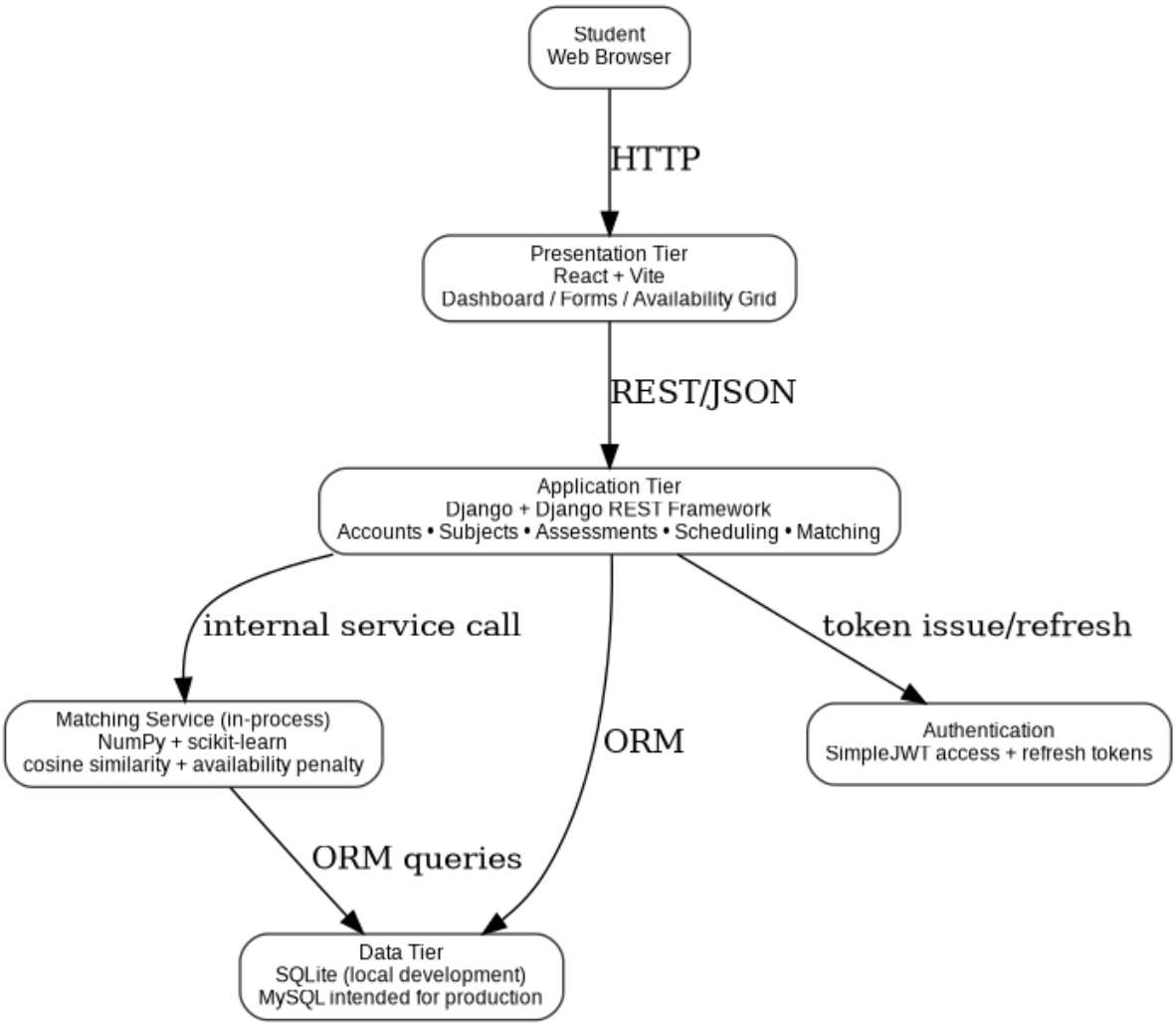
06 SSD Feedback



07 Design Class Diagram



08 System Architecture



Implementation Grounding Note

The uploaded implementation is treated as the source of truth. Earlier project documents contain planned features such as transcript parsing and invitations that are not evidenced in the current code. Those features are explicitly marked as future scope rather than being presented as implemented.